

CHAPTER 5 | Query Processing and Optimization

Query Processing, Optimization and Evaluation

Query Processing

- First stage of execution
- High level request (SQL statement) is parsed and translated into an internal representation based on relational algebra
- Identifies relations, attributes, and operations involved in the query
- Acts like a translator to convert human request to request understandable by the system

Query Processing, Optimization and Evaluation

Query Optimization

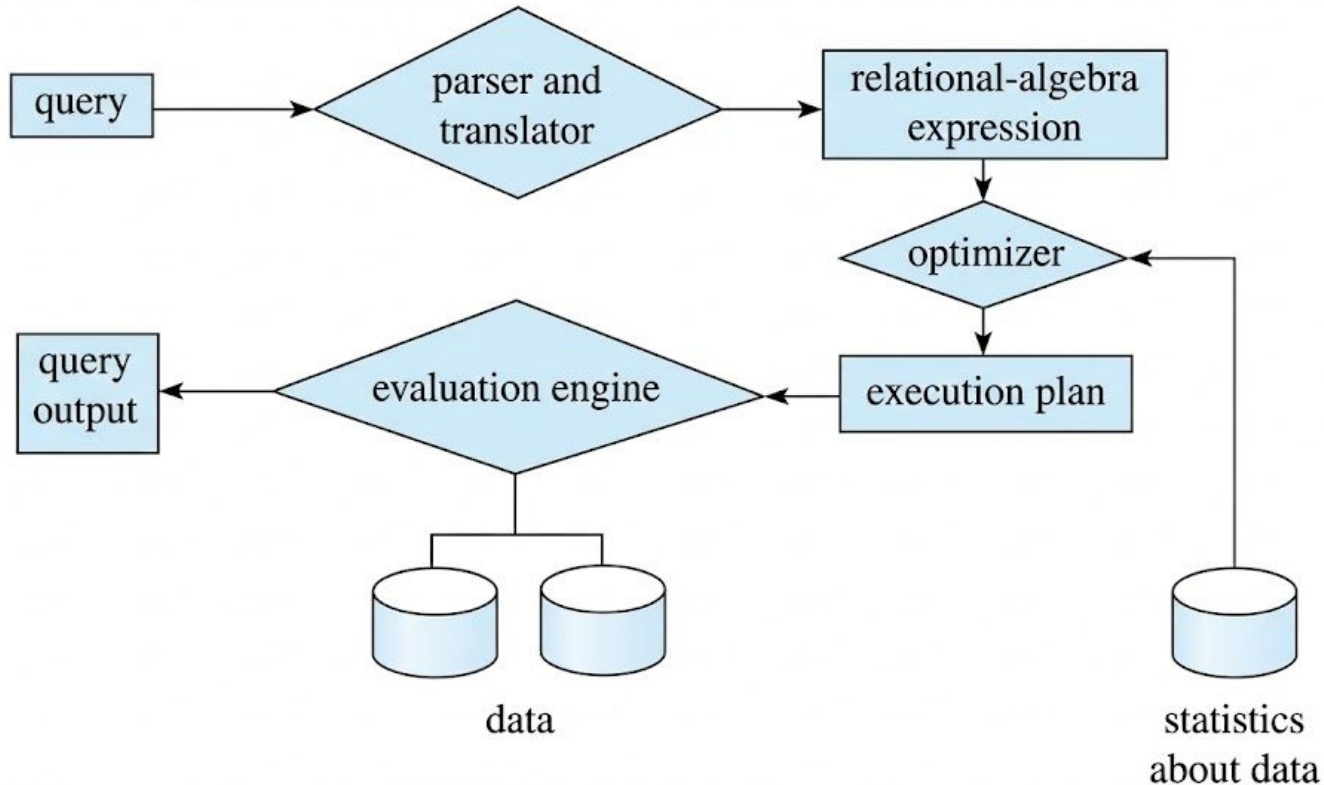
- Second stage of execution
- Analyzes multiple possible ways to execute the query
- Rewrites the query into equivalent but more efficient forms
- Chooses the most cost-effective execution plan using the optimizer
- Uses statistics and heuristics to minimize costs such as disk I/O, CPU usage, and memory consumption

Query Processing, Optimization and Evaluation

Query Evaluation

- Third stage of query processing
- Executes the query using the plan selected by the optimizer
- Involves actual access to data stored on disk or in memory
- Uses physical operations such as table scans, index scans, joins, and sorting
- Produces the final result set and returns it to the user

Block Diagram of Query Processing



Components of Block Diagram

1. Query
2. Parser and Translator
3. Relational Algebra Expression
4. Optimizer
5. Execution Plan
6. Evaluation Engine
7. Output

1. Query

This is the initial request submitted by a user or application, typically written in a high-level language like SQL. While SQL is user-friendly, it is not suitable for the system's internal use and must be converted into a machine-readable format

2. Parser and Translator

This component functions similarly to a compiler's parser. It performs scanning to identify tokens (keywords, attribute names), parsing to verify the query's syntax, and validation to ensure that all referenced relations and attributes exist in the database schema. Once validated, it translates the query into its internal representation

3. Relational Algebra

This is the internal form of the query, often represented as a query tree. It describes the query as a sequence of logical relational algebra operations (such as select, project, and join). If the query references a view, this stage replaces the view name with the algebraic expression that defines it

4. Optimizer

The optimizer explores different strategies for evaluating the query to find the one with the least estimated cost. It uses data statistics from the system catalog—such as the number of tuples in a relation and available index types—to compare the efficiency of various equivalent algebraic transformations.

4. Optimizer

Types of Optimization

a. Heuristic Optimization

This approach uses predefined rules (heuristics) to transform a query tree. It doesn't look at how much data is in the tables; it simply assumes that certain operations are "always better" if done in a specific order.

4. Optimizer

Types of Optimization

b. Cost Based Optimization

This is the most common approach in modern databases. Instead of following fixed rules, the optimizer generates multiple possible plans and assigns a "cost" (score) to each.

5. Execution Plan

Also called a query-evaluation plan, this is an annotated relational algebra tree. It specifies the evaluation primitives, which identify the exact algorithms to be used for each operation (e.g., using a specific B+-tree index for a selection or a hash join for a join operation)

6. Evaluation Engine

This runtime processor takes the query-execution plan and executes those low-level instructions against the database. It retrieves the requested data and carries out the necessary transformations defined by the plan

Query Evaluation

Query evaluation is the final execution phase of a database query. After the optimizer chooses the best "Execution Plan," the Evaluation Engine actually runs the operations (joins, scans, filters) to produce the final output.

Query Evaluation

1. Materialization

In this approach, the database evaluates one operation at a time. It computes the entire result of a sub-expression and stores it in a temporary table (materializes it) before the next operation begins.

- Pros: It is useful for complex subqueries that are used multiple times in a single query, as the result only needs to be calculated once.
- Cons: High I/O overhead. Writing large intermediate results to disk and reading them back is slow and consumes significant storage space.

Query Evaluation

2. Pipelining

Pipelining is the "streaming" version of query execution. Instead of waiting for an operation to finish, it passes individual rows (tuples) to the next operation as soon as they are ready.

- Pros: Low latency. The user starts seeing the first few rows of the result almost immediately. It also saves memory because it doesn't need to store massive intermediate tables.
- Cons: It can be difficult to implement for certain operations that require seeing "all" the data before they can produce even one row.

Performance Tuning

Performance tuning is a continuous, iterative process where a Database Administrator (DBA) or designer adjusts various aspects of a database system to ensure adequate performance as user requirements and workloads evolve.

It involves:

1. Index Tuning [EXPLAIN/ EXPLAIN ANALYZE -> ADD INDEX]
2. Conceptual Schema Tuning
3. Query and View Tuning
4. System and Transaction Tuning

Methodology	What You Tune	Typical Commands/Techniques
1. Index Tuning	Physical access paths	EXPLAIN / EXPLAIN ANALYZE → CREATE INDEX, DROP INDEX, covering indexes, included columns
2. Conceptual Schema Tuning	Table structures	Normalization, denormalization, partitioning, data types, vertical/horizontal splitting
3. Query & View Tuning	SQL statements	Rewriting queries, predicate placement, avoiding functions on indexed columns, view materialization
4. System & Transaction Tuning	Configuration & behavior	Isolation levels, lock escalation, memory allocation, I/O configuration, autocommit settings

Denormalisation for optimisation

- By storing all related data in a single physical location, the system eliminates costly JOIN operations and significantly reduces the overhead of performing multiple disk I/O operations.
- This approach saves system memory and processing power by removing the need to generate large intermediate or temporary tables during query execution.